NESNEYE DAYALI PROGRAMLAMA

Ders 10: Delegates, Events ve Windows Forms

Konular

- Delegates
- Events
- Windows Forms

Delegates (Delegeler)

- Delege bir işaretçidir.
- Bir delege referansı bir metoda olarak tutan bir referans tip değişkenidir. Referans çalışma zamanında (runtime) değiştirilebilir.
- Yordamlara aracılık eder ve onları temsil eder.
- Delegeleri, çalışma anında planda olmayan bir yordamı yürütmek için veya bir olay gerçekleştiği zaman ilgili yordamları haberdar etmek için kullanırız.
- Tüm delegeler, System.Delegate sınıfından türetilir.

Delegates (Delegeler)

- Değişkenler gibi metodlar da bellek üzerinde özel bir alanda yaşar.
- Değişken pointerleri gibi metodların başlangıç adresini işaret eden pointerler bulunur. Bunlara metod işaretçileri (function pointers) denilir. İstenirse metodlar, hafıza üzerinden sadece isimleriyle değil bu pointerler sayesinde de çağrılabilir.
- .NET, fonksiyon işaretçileriyle aynı amaca sahip tür güvenliği sağlayan delege (delegate) isimli işaretçiler sunar.

Delegates (Delegeler)

- Yazılım dilinde ise metodlara aracılık eden, onları temsil eden, onlara erişmemizi sağlayan çalışma zamanında tanımlanan yapılardır.
- Delegate, daha önceki function pointerler'den farklı olarak sadece statik metodları işaret etmez, hem statik hem de nesne (instance) metodları işaret eder.

Delegates (Delegeler)

- İki yerde ihtiyaç duyabiliriz:
  - Bazen çalışma zamanında planda olmayan bir metodu yürütmek zorunda kalabiliriz. Bunun için delegateler kullanılır.
  - Bir olay(event) gerçekleştiği zaman bir veya daha fazla metodu, gerçekleşen olaydan haberdar etmek için delegeler kullanırız. Bu sayede, delegeler, eventlerle anlam kazanır.

Delege İşlemler

- Bir delege işlemi üç adımdan oluşur.
    1. Tanımlama(Declaration)
    2. Örnekleme, gerçeklenme (Instantiation)
    3. Çağırma(Invocation)

Delege tanımlama

- Delege tanımlama, delege tarafından refere edilecek metodu belirtmektir.
- Delege sadece aynı imzaya (yapıya - signature) sahip bir metodu refere edebilir.

```
public delegate int MyDelegate (string s);
```

```
delegate <return type> <delegate-name> <parameter list>
```

delegate anahtarından sonra döndüreceği değer, delege ismi ve delege'e ait parametre sayısı ve türüne girilir. Bu ifade bir imzadır. (signature)

Delege Örnekleme ve Çağırma

```
class Program
{
    //1.tanımlama
    public delegate int MyDelegate(int x);

    static int KareAl(int sayi)
    {
        return sayi * sayi;
    }

    static void Main(string[] args)
    {
        MyDelegate delege1 = new MyDelegate(KareAl); //2.declaration
        int y = delege1(4); //3.Invocation

        Console.WriteLine(y);

        Console.ReadKey();
    }
}
```

- KareAl fonksiyonu ile Delege'nin parametre sayısı / türü ve döndürdüğü değerler aynıdır.
- Delege üzerinden New bir degele yaratılıp KareAl fonksiyonuna bağlanır.
- Delege üzerinden KareAl fonksiyonu kullanıma hazırdır.

Multicasting

- Bir delegeyi C veya C++'taki function pointerlerden farklı olarak birden fazla metoda temsilci kılabiliriz. Buna çoklu temsilcilik (Multicasting) denilir.
- Delegenin listesine yeni bir metod eklemek için C#'ta "+=" operatörü kullanılır.
- Listeden bir metod çıkarmak için C#'ta "-=" operatörü kullanılır.
- Bir delegeyle ilişkilendirilmiş metodlar o delegenin çağrı listesinde yer alır. Bu yüzden delegeyle ilişkilendirilmiş metodlara erişmek için delegenin InvocationList() metodu kullanılır.

Multicasting

```
public delegate void MyDelegate(int Deger1, int Deger2);

void Toplama(int Sayi1, int Sayi2)
{
    Console.WriteLine("{0} + {1} = {2}", Sayi1, Sayi2, Sayi1 + Sayi2);
}//Toplama

void Carpma(int Sayi1, int Sayi2)
{
    Console.WriteLine("{0} * {1} = {2}", Sayi1, Sayi2, Sayi1 * Sayi2);
}//Carpma

void Bolme(int Sayi1, int Sayi2)
{
    Console.WriteLine("{0} / {1} = {2}", Sayi1, Sayi2, Sayi1 / Sayi2);
}//Bolme
```

- Bir delege ve delege özelliklereine sahip üç fonksiyon vardır. Fonksiyonlar diğer örnekten farklı olarak static değildir.

Multicasting

```
public Program()
{//Constructor
    MyDelegate delege = null;
    delege += new MyDelegate(Toplama);
    delege += new MyDelegate(Carpma);
    delege += new MyDelegate(Bolme);

    int X, Y;
    X = 18; Y = 3;
    delege(X, Y);
    Delegate[] DlgList = delege.GetInvocationList();
    Console.WriteLine("Delegenin temsil ettiği metodlar :");
    Console.WriteLine("---------------------------------");
    for (byte i = 0; i < DlgList.Length; i++)
        Console.WriteLine(DlgList[i].Method.Name);

    delege -= new MyDelegate(Bolme);
    int X1, Y1;
    X1 = 30; Y1 = 5;
    delege(X1, Y1);

    DlgList = delege.GetInvocationList();
    Console.WriteLine("Delegenin temsil ettiği metodlar :");
    Console.WriteLine("---------------------------------");
    for (byte i = 0; i < DlgList.Length; i++)
        Console.WriteLine(DlgList[i].Method.Name);
}//Program
```

Delegeye metot ekleme

Mevcut metodun özelliklerini çağırma

Delegeye metot çıkarma

- Program sınıfına ait Program Constructor'ı içine multicasting işlemler yapılmıştır.
- GetInvocationList fonsiyonu dizi olarak Delege listesi döndürür. Bu fonksiyon ile delegeye ait metotları görebiliriz. Bu örnekte for döngüsü ile delegenin içerdiği metotların isimleri döndürülmüştür.

Anonymous Methods (İsimsiz Metotlar)

- Delegelerin aynı signature sahip herhangi bir metodu refere ederek kullanır. Başka bir deyişle, bir delege tarafından refere edilebilen bir metot çağrılabilir.
- İsimsiz metotlar bir delege parametresi olarak bir kod bloğuna geçirmek için bir teknik sağlar.
- İsimsiz metotlar sadece code bölümü vardır, ad bölümü ise yoktur.
- İsimsiz bir metotta değer döndürme tipini belirlemeye ihtiyaç yoktur. Ama İsimsiz bir metoda isimli bir metot bağlanabilir.

Anonymous Methods

```
delegate void NumberChanger(int n);
...
NumberChanger nc = delegate(int x)
{
   Console.WriteLine("Anonymous Method: {0}", x);
};
```

- Void bir delege çağrılmış. Çağırma işleminde metodun bir ismi yok.
- Console.WriteLine ile çıktı ekrana çıktı alınmış.

Anonymous Methods

```
delegate void NumberChanger(int n);

class TestDelegate
{
    static int num = 10;
    public static void SayiEkle(int p)
    {
        num += p;
        Console.WriteLine("İsimli Method: {0}", num);
    }

    public static void SayiCarp(int q)
    {
        num *= q;
        Console.WriteLine("İsimli Method: {0}", num);
    }

    public static int getSayi()
    {
        return num;
    }

    static void Main(string[] args)...
}
```

```
static void Main(string[] args)
{
    //İsimsiz bir metot kullanarak delegeye bağlanıyor.
    NumberChanger nc = delegate(int x)
    {
        Console.WriteLine("Anonymous Method: {0}", x);
    };

    //İsimsiz metoda değer gönderiliyor
    nc(10);
    //İsimli bir metot isimsiz metoda bağlanıyor.
    nc = new NumberChanger(SayiEkle);
    //İsimsiz metoda değer gönderiliyor.
    nc(5);
    //İsimli bir metot isimsiz metoda bağlanıyor.
    nc = new NumberChanger(SayiCarp);
    //İsimsiz metoda değer gönderiliyor.
    nc(2);
    Console.ReadKey();
}
```

Değişkensiz nesne oluşturma

- Static bir Main olmadan programımızı çalıştıramayız. Tüm kodlarımızı Program sınıfı içine yazdık ve yazdığımız metotlar hiç biri static değildir.
- Program constructor'ını çağırırsak programı çalıştırıabilir.
- Program constructor'ı ancak bir nesne oluşturarak çağırabiliriz.
- Nesne oluşturduktan sonra nesnenin döndürdüğü sınıfı kullanmayacaksak sadece new Sınıf_Adı dememiz yeterlidir.

```
static void Main(string[] args)
{
    //Program _p_nesne = new Program();
    //eğer sadece constructor kullanılacaksa
    new Program();
    Console.ReadKey();
}
```

Events (Olaylar)

- Olaylar: klavye'de bir tuşa basma, fareye tıklama, fare hareketleri, vb. kullanıcı hareketleridir.
- Uygulamalar, bu hareketler meydana geldiği zaman olaylara cevap vermeye ihtiyaç duyar.
- Olaylar inter-process iletişimi kullanır. (Interrupt)
- Özellikle bundan sonraki göreceğimiz Windows Forms konusunda olaylar karşımıza çıkacak.

Delegeler ile event'ları kullanma

- Olaylar, bir sınıfta ve aynı sınıf / farklı sınıftaki kullanılan delegelerle olay yöneticileri ile tanımlanır ve hareket geçer.
- Olay içeren bir sınıfa publisher (yayıncı) sınıf denir.
- Diğer sınıflar bu olayı kullanabilir. Bu durumda bu sınıflar subscriber (abone) sınıfı olur.

```
public delegate void BoilerLogHandler(string status);
```

```
public event BoilerLogHandler BoilerEventLog;
```

Delege tanımlamandıktan sonra event'a event anahtar sözcüğünden sonra delege yukarıdaki gibi bağlanır.

Events

Delege olay bağlantısı yapıldı.

Olay çağrıldı

Olay tetiklendi

```
public class OlayTest
{
    private int value;
    public delegate void NumManipulationHandler();
    public event NumManipulationHandler ChangeNum;
    protected virtual void OnNumChanged()
    {
        if (ChangeNum != null)
        {
            ChangeNum();
        }
        else
        {
            Console.WriteLine("Olay başladı!");
        }
    }

    public OlayTest(int n)
    {
        SetDeger(n);
    }

    public void SetDeger(int n)
    {
        if (value != n)
        {
            value = n;
            OnNumChanged();
        }
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        OlayTest e = new OlayTest(5);
        e.SetDeger(7);
        e.SetDeger(11);
        Console.ReadKey();
    }
}
```

```
Event Fired!
Event Fired!
Event Fired!
```

Event

```
class Program
{
    public delegate void EventHandler();
    public static event EventHandler _show;

    static void Main()
    {
        //Birden fazla olayı _show olayına ekleme
        _show += new EventHandler(Dog);
        _show += new EventHandler(Cat);
        _show += new EventHandler(Mouse);
        _show += new EventHandler(Mouse);

        //Olayı çağırma
        _show.Invoke();
    }
    static void Cat()
    {
        Console.WriteLine("Cat");
    }
    static void Dog()
    {
        Console.WriteLine("Dog");
    }
    static void Mouse()
    {
        Console.WriteLine("Mouse");
    }
}
```

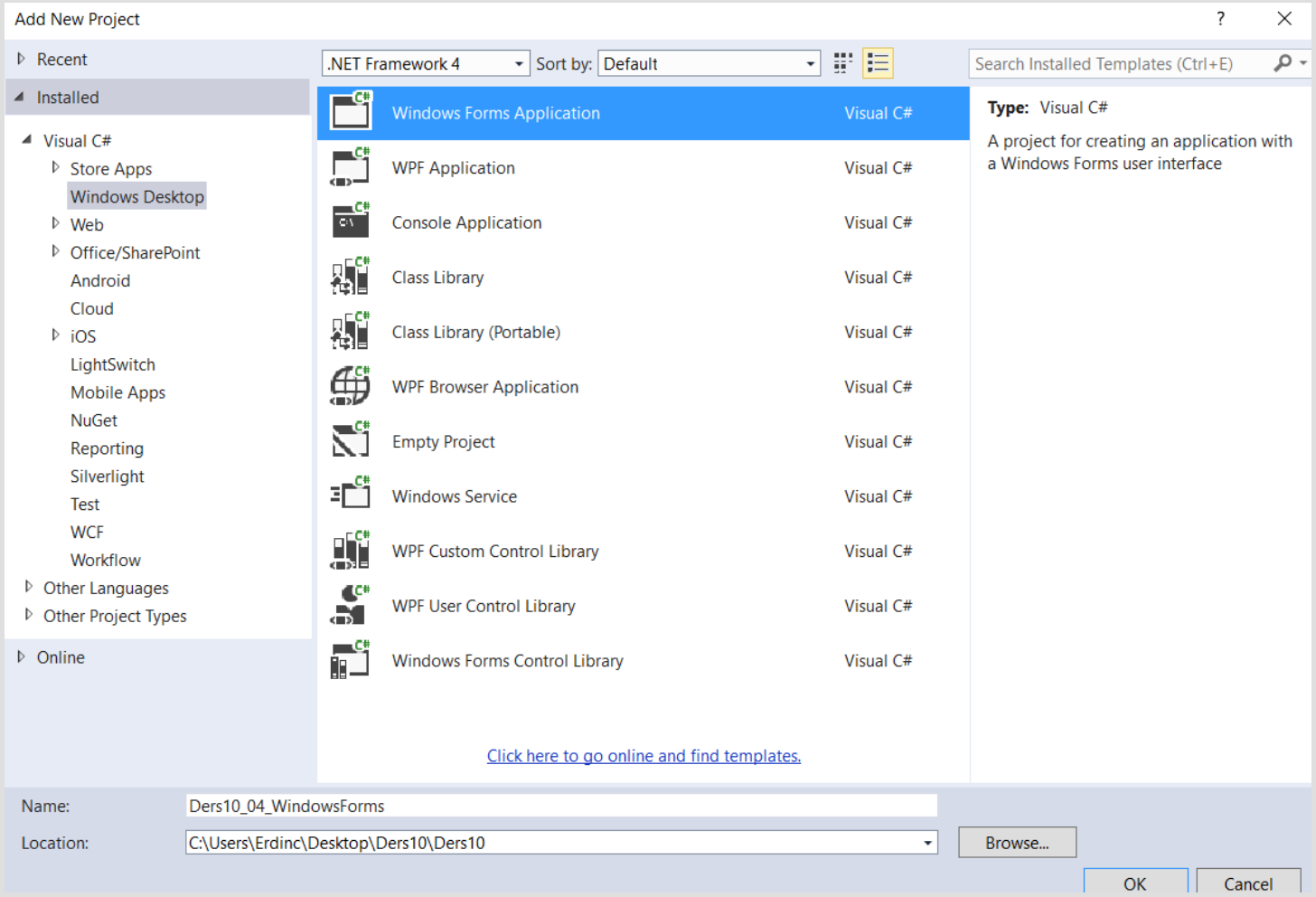
- Bir olay birçok işleyiciye sahip olabilir.
- Olay işleyici sayesinde birçok metotu bir olaya bağlayabiliriz. (Ekleme Multicasting konusuna benzer hatta -= çıkartabiliriz.)
- Invoke fonksiyonu ile olay tetiklenir ve Mouse fonksiyonu iki defa olmak üzere tüm fonksiyonlar çağrılır.

Windows Forms

- Button, Pencere, Text (Metin), DataGridView (Satır ve Sütunlardan oluşan tablo) ve bir çok bileşenden oluşan bir programdır.
- Windows Forms geliştiricinin işini kolaylaştırır.
- Sürekle-bırak (drag-drop) olayına destek verdiği için tasarımı da kolaylaştırır.
- GUI (Graphical User Interface – Grafiksel Kullanıcı Arayüzü) kolay şekilde yapmamıza olanak sağlar.

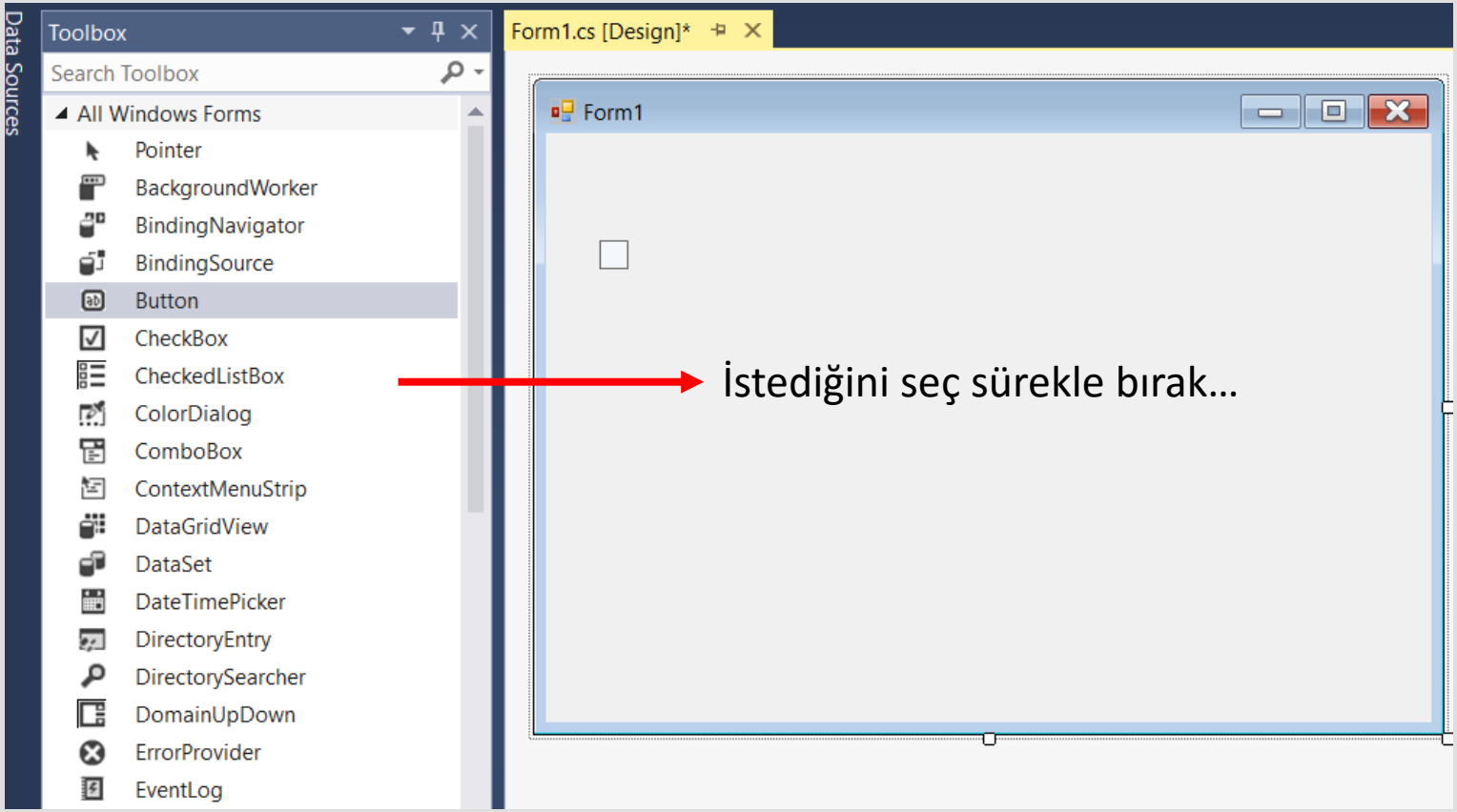
Windows Forms

- Öncelikle Yeni bir Proje oluşturulurken Console Application yerine Windows Forms Application'ı seçelim. Karşımıza bir pencere gelecek.



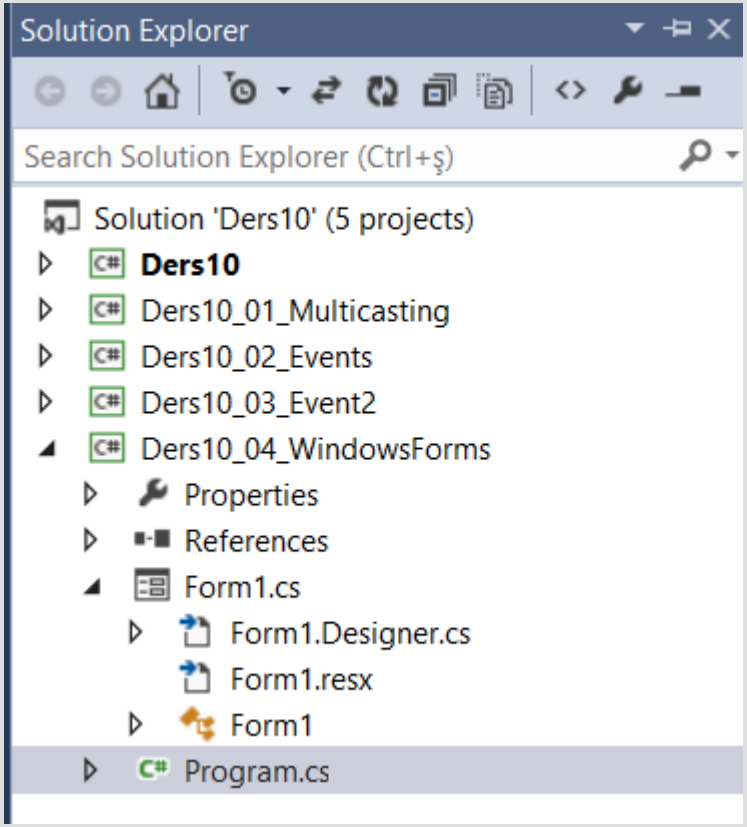
Windows Forms

- Genelde sol bölümde bulunan ToolBox sekmesine tıklayalım. Karşımıza birçok bileşen (component) geldi. Component'ler Windows Forms için geliştirilmiş sınıflardır. Component'ler sayesine hızlıca görsel bir bileşen oluşturabiliriz.

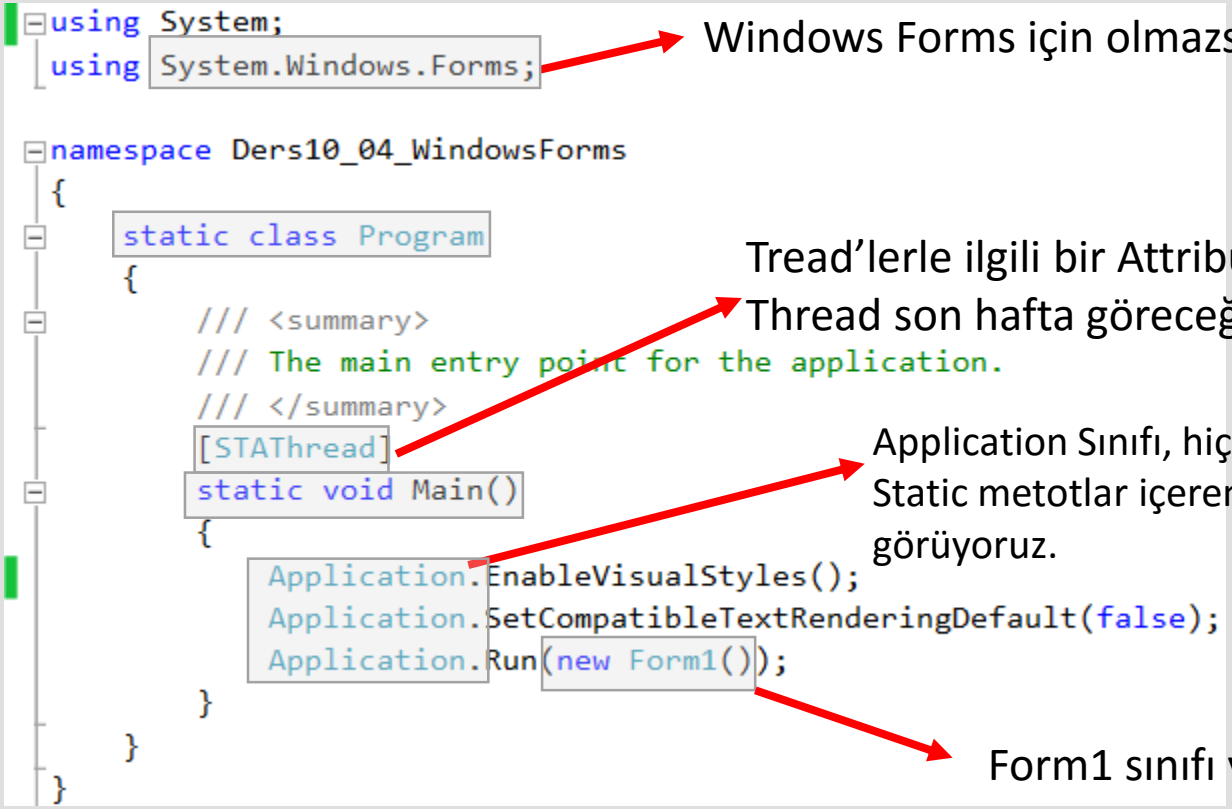


Windows Forms

- Şu ana kadar hiç kod görmedik!!! İmkansız, sizin yaptığınız her işlem arka planda koda çevrilmek zorundadır. Hatta .Net Nesneye Yönelik bir dil olduğu için arka planda sınıf ve nesne (new etiketi) bolca olması lazım. Şimdi Arka plandaki çalışmayı inceleyelim.
- Solution Explorer'da oluşan önemli dosyaları tekere teker inceleyelim.
  - Program.cs
  - Form1.cs
  - Form1.Designer.cs



Windows Forms – Program.cs



- Program sınıfı static tanımlanmış ve Main static metoduna sahip. Aslında Console uygulamasına çok benziyor. Static bir main olmadan program başlatılamıyor.

Windows Forms – Form1. cs

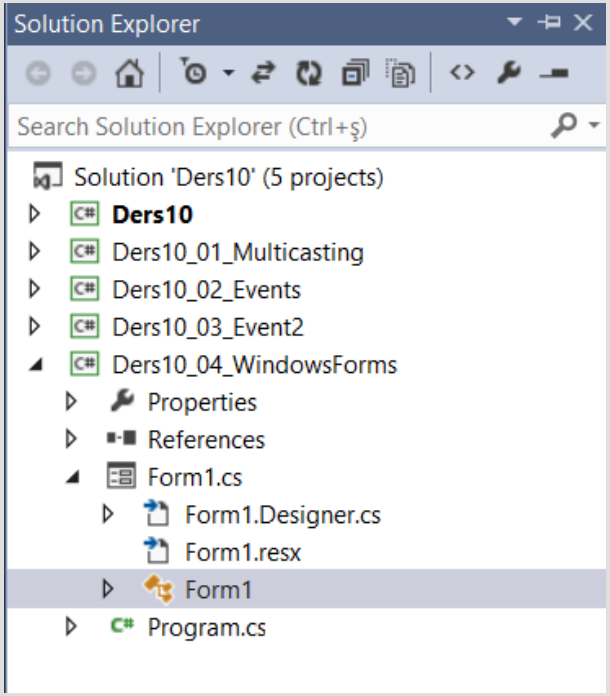
- Form1'in kod içeriğini ve cs dosyasını görmek için Form1 üerine tıklayalım. Eğer Form1.cs tıklarsak görsel pencere ekranı karşımıza çıkar.

Form1.cs | Form1.Designer.cs | Program.cs

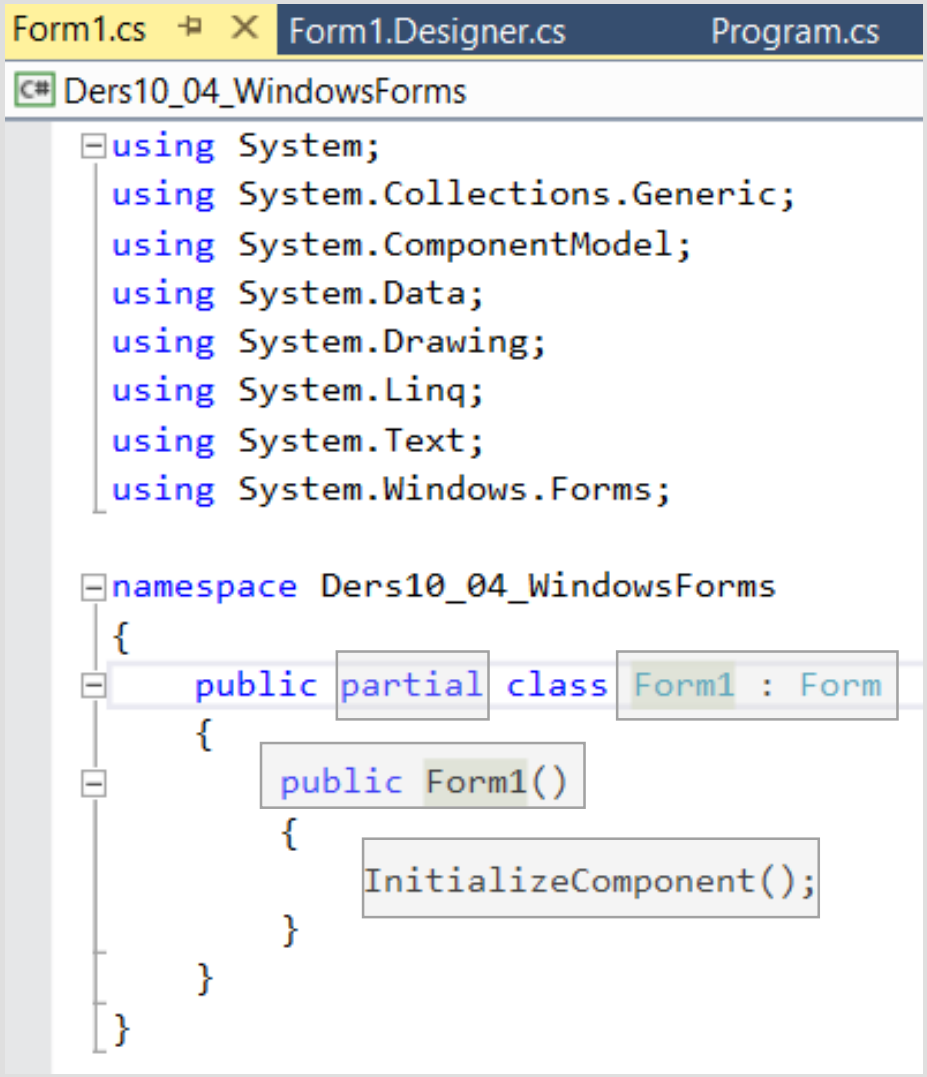
Ders10_04_WindowsForms

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;

namespace Ders10_04_WindowsForms
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }
    }
}
```



Windows Forms – Form1. cs



- Using yapılmış bir çok namespace var. Şu anlık bunların Form işlemlerinde olduğunu kabul edelim. Zaten otomatik olarak oluşturuldu.
- Form1'in kod içeriğini ve cs dosyasını görmek için Form1 üerine tıklayalım. Eğer Form1.cs tıklarsak görsel pencere ekranı karşımıza çıkar.
- Partial bir Form1 sınıfı ile karşılaştık. Demekki bu sınıfın başka bir yerde bir parçası olabilir.
- Form1 ayrıca Form sınıfını base olarak kullanıyor. Kalıtım konusunu hatırlattı.
- Public Form1() constructor yine karşımıza çıktı. Program.cs içindeki new Form1() demekki bu constructor'ı tetikliyor.
- InitializeComponent fonksiyonu var. Direkt çağrıldığına göre Sınıf içinde olması gerekir. Görünürde yok ama partial class olduğunu unutmayalım.

Windows Forms – Form1.Designer.cs

Form1.cs | Form1.Designer.cs | Program.cs | Form1.cs [Design]

Ders10_04_WindowsForms | Ders10_04_WindowsForms.Form1

```
namespace Ders10_04_WindowsForms
{
    partial class Form1
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components = null;

        /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources should be disposed; otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        Windows Form Designer generated code
    }
}
```

- Partial class Form1'in parçasını bulduk. Aynı namespace içinde bulunuyor.
- Bir private değişken components isminde null durumda görünüyor.
- Dispose base Form1'de bir metot olduğunu tahmin ediyoruz çünkü override edilmiş. Dispose component'leri yerleştirme işlemidir. Components değişkeni dispose yapılmıştır.
- Windows Form Designer generated code region'ını görüyoruz. Sürükle bırak yapıldığında otomatik olarak oluşan bilgiler buraya gidiyor.

Windows Forms – Form1.Designer.cs

```
#region Windows Form Designer generated code

/// <summary>
/// Required method for Designer support - do not modify
/// the contents of this method with the code editor.
/// </summary>
private void InitializeComponent()
{
    this.SuspendLayout();
    //
    // Form1
    //
    this.AutoScaleDimensions = new System.Drawing.SizeF(8F, 16F);
    this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
    this.ClientSize = new System.Drawing.Size(612, 407);
    this.Name = "Form1";
    this.Text = "Form1";
    this.ResumeLayout(false);

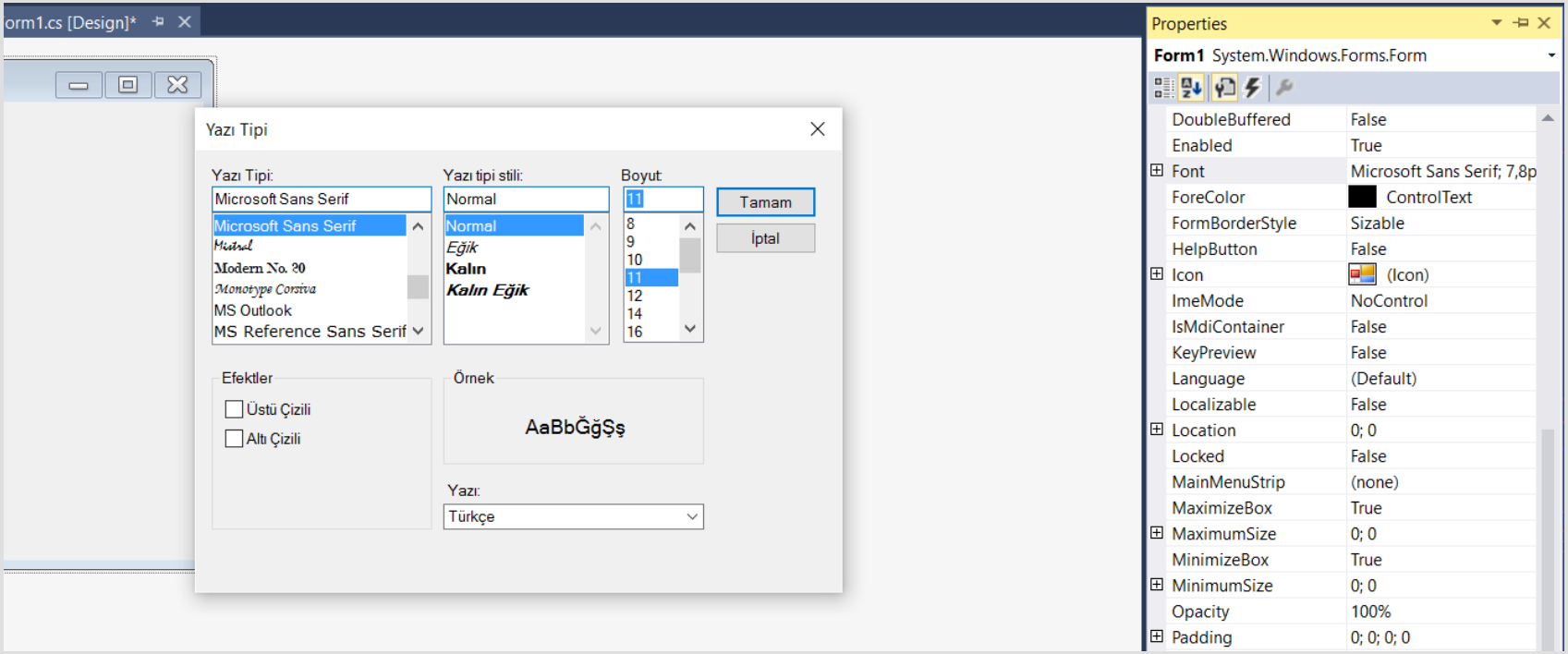
}

#endregion
```

- this Form1'i işaret ediyor.
- InitializeComponent metodunu bulduk. Form1 içinde olması gerektiğini söylemiştik.
- Form'a ait bazı ayarlarını yapıldığını görüyoruz. Bazı ayarlar nesne bilgi istediğini görüyoruz. (new kullanılan satırlar.)
- Ayrıca .Name ve .Text bilgilerini içeren satırlarda, .Name ve Text'in string türünden set yordamı bulunan birer property olduğunu görüyoruz.

Windows Forms - Property

- Tasarım ekranını açıp sağ tuş ve properties diyelim. Karşımıza Form1 için olan tüm ayarlar gelecektir. Font bölümüne tıklayalım. İstediğimiz gibi bir değişiklik yapalım.



Windows Forms - Property

- Font ayarını değiştirdiğinizde InitializeComponent değişikliğe dikkat edin.
- Property bölümünde Form'a ait birçok property olduğunu gördük fakat bu kısımda sadece bir kısmı var. Bunun sebebi kalıtımdır. Form1'i Form sınıfında türetmiştik. Form sınıfında default ayarlar tutulur. Sadece değişen ayarlar bu bölüme yazılır. İşte NYP güzel tarafları…

```
private void InitializeComponent()
{
    this.SuspendLayout();
    //
    // Form1
    //
    this.AutoScaleDimensions = new System.Drawing.SizeF(11F, 27F);
    this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
    this.ClientSize = new System.Drawing.Size(853, 487);
    this.Font = new System.Drawing.Font("Arial Narrow", 13.8F, System.Drawing.FontStyle.Regular, System.Drawing.GraphicsUnit.Point, ((byte)(162)));
    this.Margin = new System.Windows.Forms.Padding(4, 5, 4, 5);
    this.Name = "Form1";
    this.Text = "Form1";
    this.ResumeLayout(false);

}
```

Windows Forms - Property

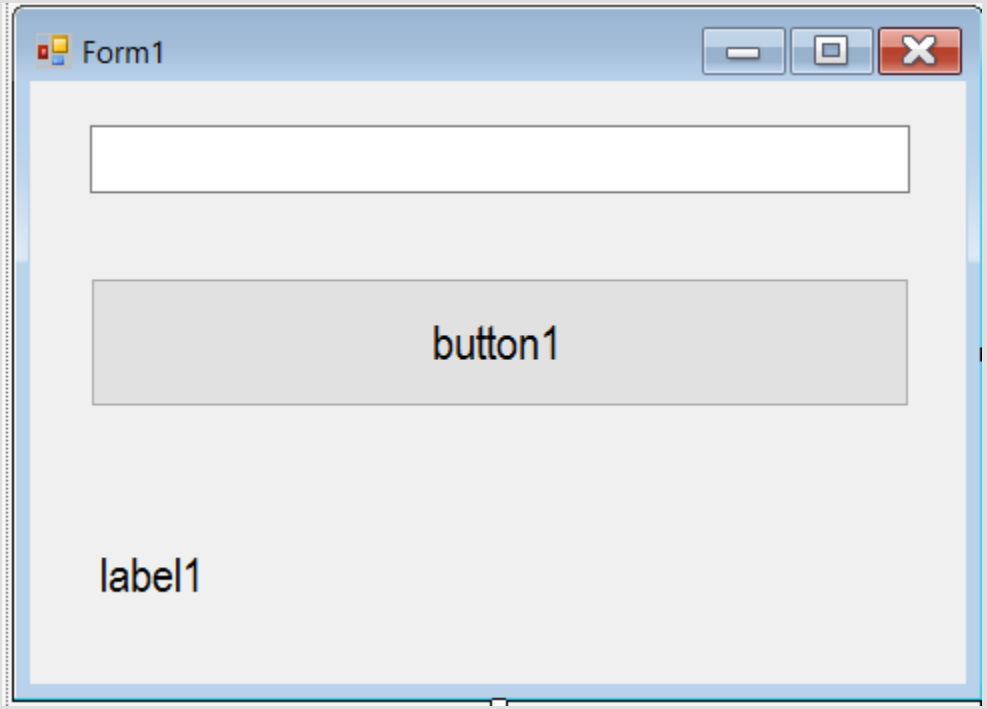
- Font ayarını değiştirdiğinizde InitializeComponent değişikliğe dikkat edin.
- Property bölümünde Form'a ait birçok property olduğunu gördük fakat bu kısımda sadece bir kısmı var. Bunun sebebi kalıtımdır. Form1'i Form sınıfında türetmiştik. Form sınıfında default ayarlar tutulur. Sadece değişen ayarlar bu bölüme yazılır. İşte NYP güzel tarafları…

```
private void InitializeComponent()
{
    this.SuspendLayout();
    //
    // Form1
    //
    this.AutoScaleDimensions = new System.Drawing.SizeF(11F, 27F);
    this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
    this.ClientSize = new System.Drawing.Size(853, 487);
    this.Font = new System.Drawing.Font("Arial Narrow", 13.8F, System.Drawing.FontStyle.Regular, System.Drawing.GraphicsUnit.Point, ((byte)(162)));
    this.Margin = new System.Windows.Forms.Padding(4, 5, 4, 5);
    this.Name = "Form1";
    this.Text = "Form1";
    this.ResumeLayout(false);

}
```

Windows Forms

- Tasarım ekranını açıp sürükle bırak ile bir Button, bir Label ve bir de Text oluşturalım.



Windows Forms

- Label, Text ve Button eklendikten sonra InitializeComponent metodundaki değişimleri inceleyelim.
- New ile üçününde oluşturulduğunu görüyoruz. Üçü de kendine ait sınıflardan yaratılmışlar. Fakat button1, textbox1 ve label1 değişken olarak tanımlanmalıydı. Koda tekrar baktığımızda hemen region sonunda 3'ün de private olarak tanımlandığını görüyoruz.
- Microsoft .Net tamamen NYP uyumlu geliştirmiştir. Sizin yaptığınız her sürükle bırak işlemi NYP'da bir karşılığı vardır.

```
#endregion

private System.Windows.Forms.Button button1;
private System.Windows.Forms.TextBox textBox1;
private System.Windows.Forms.Label label1;
```

```
this.button1 = new System.Windows.Forms.Button();
this.textBox1 = new System.Windows.Forms.TextBox();
this.label1 = new System.Windows.Forms.Label();
this.SuspendLayout();
//
// button1
//
this.button1.Location = new System.Drawing.Point(30, 98);
this.button1.Name = "button1";
this.button1.Size = new System.Drawing.Size(410, 65);
this.button1.TabIndex = 0;
this.button1.Text = "button1";
this.button1.UseVisualStyleBackColor = true;
//
// textBox1
//
this.textBox1.Location = new System.Drawing.Point(30, 22);
this.textBox1.Name = "textBox1";
this.textBox1.Size = new System.Drawing.Size(410, 34);
this.textBox1.TabIndex = 1;
//
// label1
//
this.label1.AutoSize = true;
this.label1.Location = new System.Drawing.Point(30, 232);
this.label1.Name = "label1";
this.label1.Size = new System.Drawing.Size(65, 29);
this.label1.TabIndex = 2;
this.label1.Text = "label1";
```

Windows Forms

- Her ne kadar nesneleri yaratmış olsakta şu an Form1 üzerinde nesnelerimiz gözükmez. Bunun için Form1 bölümüne Controls.Add ile 3 nesnemizde eklenmiştir.
- Bu sayede Form1 ile yeni oluştulan nesneler ilişkilendirilmiş olur.

```
//
// Form1
//
this.AutoScaleDimensions = new System.Drawing.SizeF(11F, 27F);
this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
this.ClientSize = new System.Drawing.Size(468, 301);
this.Controls.Add(this.label1);
this.Controls.Add(this.textBox1);
this.Controls.Add(this.button1);
this.Font = new System.Drawing.Font("Arial Narrow", 13.8F, System.Drawing.FontStyle.Regular, System.Drawing.GraphicsUnit.Point, ((byte)(162)));
this.Margin = new System.Windows.Forms.Padding(4, 5, 4, 5);
this.Name = "Form1";
this.Text = "Form1";
this.ResumeLayout(false);
this.PerformLayout();
```

Windows Forms - Events

- Button üzerine tıklayalım. Karşımıza bir metot çıktı fakat bu metot nasıl çalıştı.
- Bu metot içine aşağıdaki gibi bir kod yazalım.
- Şu an yaptığımız işlem textBox1 e ne yazılıyorsa button1'e tıklandığında git Label1 içine yaz. Hem textbox1 hem de Label1 için Text property özelliğini kullanılıyoruz. Text hem set hem de get yapılabilen bir property'dir ve nesnenin metnini değiştirmede kullanılır. (Component'lerde bu tür birçok property vardır. Bu property'leri Windows Forms ortamında kod yazdıkça göreceksiniz. Bu dersin amacı NYP tarafını anlatmaktır. NYP çözüldüğünde ileri de Windows Forms gitse Windows X gibi yeni bir ortam gelse de kurallar değişmez.)
- Aşağıdaki kodta label1.Text üzerinden get işlemi yapılırken textBox1 için ise set işlemi yapılmaktadır.

```
private void button1_Click(object sender, EventArgs e)
{
    textBox1.Text = label1.Text;
}
```

Windows Forms - Events

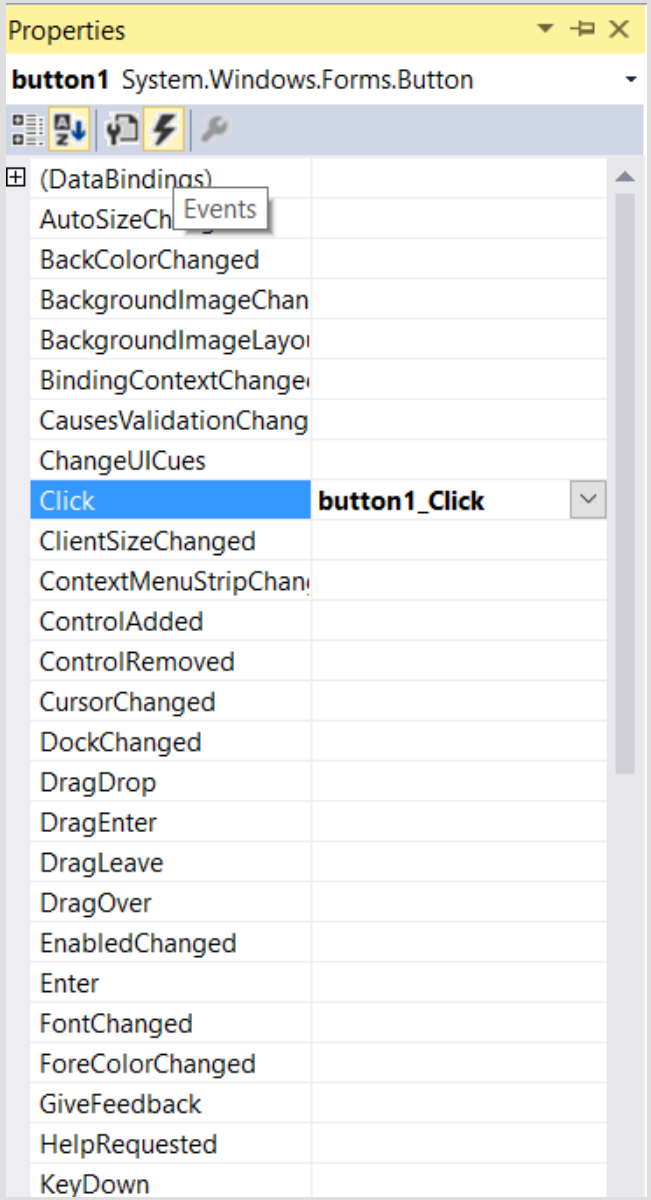
- Button1 click olayı için InitializeComponent metodundaki Button1 bölümündeki değişimleri inceleyelim.

```
//
// button1
//
this.button1.Location = new System.Drawing.Point(30, 98);
this.button1.Name = "button1";
this.button1.Size = new System.Drawing.Size(410, 65);
this.button1.TabIndex = 0;
this.button1.Text = "button1";
this.button1.UseVisualStyleBackColor = true;
this.button1.Click += new System.EventHandler(this.button1_Click);
```

- Operatör overloading, Delege Multicasting ve eevents konuları aklımıza gelmiştir. Click metodu içine += ile yeni bir EventHandler ekleniyor. EventHandler bir önceki slaytta yazdığımız metodun ismini içeriyor.

Windows Forms - Events

- Bir birleşen üzerine gelip sağ tuşu tıklayıp Property Bölümünde «şimşek» şeklindeki ikona tıkladığımızda o bileşen için tüm olayları görüntüleyebiliriz.
- Olayları metotlarımıza bağlayabilir ve o olay üzerine tek tıklayarak olaya ait bir metot açabiliriz.
- Her bileşenin kendine has olayları olabileceğini unutmayın. Örnekte sadece Button olayına ait olaylar vardır ve bu olaylardan sadece Click olayı aktiftir.



Kaynaklar

- http://www.tutorialspoint.com/csharp/
- https://msdn.microsoft.com/en-us/library/ms229601(v=vs.110).aspx